Modeling Styles in Verilog

RTL Design Fundamentals

October 12, 2026

Contents

1 Introduction **2**

2 Gate-Level Modeling **2**
2.1 Example: AND Gate 2

3 Dataflow Modeling **2**
3.1 Example: Half Adder 3

4 Behavioral Modeling **3**
4.1 Example: 2-to-1 Multiplexer 3

5 Sequential Behavioral Modeling **4**
5.1 Example: D Flip-Flop 4

6 Structural Modeling **4**
6.1 Example: Half Adder Using Gate Instances 5

7 Comparison of Modeling Styles **5**

8 RTL Design Guidelines **5**

9 Summary **6**

1 Introduction

Verilog supports different modeling styles to describe digital hardware. The major styles are:

1. Gate-level modeling
2. Dataflow modeling
3. Behavioral modeling
4. Structural modeling

These styles differ in how the hardware functionality and connections are represented. They can be combined within a single design.

2 Gate-Level Modeling

Gate-level modeling describes a circuit using built-in logic gate primitives such as `and`, `or`, `not`, and `xor`.

2.1 Example: AND Gate

```
module and_gate (
    input  wire a,
    input  wire b,
    output wire y
);

    and g1 (y, a, b);

endmodule
```

Explanation:

- The primitive `and` represents an AND gate.
- `g1` is the instance name.
- The output is connected to `y`.

$$y = a \wedge b$$

Application: Useful for illustrating logic gates and describing small gate-level networks.

3 Dataflow Modeling

Dataflow modeling describes the relationship between inputs and outputs using continuous assignments.

The `assign` statement is commonly used with expressions involving logical, bitwise, and arithmetic operators.

3.1 Example: Half Adder

```
module half_adder (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);

    assign sum   = a ^ b;
    assign carry = a & b;

endmodule
```

Explanation:

- Sum is generated using XOR.
- Carry is generated using AND.
- The assignments continuously reflect changes in the input signals.

$$\text{Sum} = a \oplus b$$

$$\text{Carry} = a \wedge b$$

Application: Suitable for combinational logic such as adders, multiplexers, and Boolean expressions.

4 Behavioral Modeling

Behavioral modeling describes what a circuit does using procedural statements such as `if`, `case`, and assignment statements.

The `always` block is commonly used to model combinational and sequential logic.

4.1 Example: 2-to-1 Multiplexer

```
module mux2to1 (
    input  wire a,
    input  wire b,
    input  wire sel,
    output reg  y
);

    always @(*) begin
        if (sel)
            y = b;
        else
            y = a;
```

```
    end

endmodule
```

Explanation:

- If `sel = 0`, the output is `a`.
- If `sel = 1`, the output is `b`.
- All output paths are assigned, so no latch is inferred.

$$y = \begin{cases} a, & sel = 0 \\ b, & sel = 1 \end{cases}$$

Application: Useful for control logic, multiplexers, counters, and finite state machines.

5 Sequential Behavioral Modeling

Clocked behavioral blocks describe registers and other sequential logic.

5.1 Example: D Flip-Flop

```
module d_flip_flop (
    input  wire clk,
    input  wire d,
    output reg  q
);

    always @(posedge clk) begin
        q <= d;
    end

endmodule
```

Explanation:

- The process executes on the rising edge of the clock.
- The output captures the input value at that clock edge.
- The nonblocking assignment models the register update.

Application: Used for registers, counters, pipelines, and state machines.

6 Structural Modeling

Structural modeling describes a circuit by instantiating smaller modules and connecting them together.

6.1 Example: Half Adder Using Gate Instances

```
module half_adder_struct (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);

    xor g1 (sum, a, b);
    and g2 (carry, a, b);

endmodule
```

This example instantiates two gate primitives. Larger structural designs can instantiate reusable modules and connect their ports using wires.

Application: Useful for hierarchical designs, module integration, and designs with clearly defined submodule boundaries.

7 Comparison of Modeling Styles

Style	Description	Typical Use
Gate-level	Uses logic gate primitives	Small gate networks
Dataflow	Uses continuous assignments and expressions	Combinational equations
Behavioral	Uses procedural statements	Control and sequential logic
Structural	Connects instantiated components	Hierarchical designs

8 RTL Design Guidelines

- Prefer clear dataflow or behavioral RTL for ordinary combinational logic.
- Use nonblocking assignments in clocked sequential blocks.
- Use blocking assignments in combinational procedural blocks.
- Assign all combinational outputs on every path to avoid unintended latches.
- Use structural modeling when module hierarchy or explicit component connections are needed.
- Check synthesis results to confirm that the intended hardware is inferred.

9 Summary

- **Gate-level:** Describes logic using gate primitives.
- **Dataflow:** Describes relationships using continuous assignments.
- **Behavioral:** Describes functionality using procedural statements.
- **Structural:** Describes hardware by connecting modules or components.

These modeling styles are complementary. RTL design engineers choose a suitable style based on the design requirements, readability, reuse, and synthesis goals.